🟢 Easy Level (1–10 with solutions)

---

1. Reverse a String

Problem: Reverse `"hello"` without slicing.

✅ **Answer:**

```
s = "hello"
rev = ""
for ch in s:
    rev = ch + rev   # add character in front
print(rev)
```

Output:

```
olleh
```

---

2. Count Vowels

Problem: Count vowels in `"programming"`.

✅ **Answer:**

```
s = "programming"
vowels = "aeiou"
count = 0
for ch in s:
    if ch in vowels:
        count += 1
print("Vowel count:", count)
```

Output:

```
Vowel count: 3
```

---

3. Even or Odd

Problem: Check if number is even or odd.

✅ **Answer:**

```
num = 7
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Output:

```
Odd
```

---

4. Sum of List

Problem: Find sum of `[1,2,3,4]` without `sum()`.

✅ **Answer:**

```
nums = [1, 2, 3, 4]
total = 0
for n in nums:
    total += n
print(total)
```

Output:

```
10
```

---

5. Print Numbers with Break & Continue

Problem: Print 1–20, skip multiples of 3, stop if number is 15.

✅ **Answer:**

```
for i in range(1, 21):
    if i % 3 == 0:
        continue
    if i == 15:
        break
    print(i, end=" ")
```

Output:

```
1 2 4 5 7 8 10 11 13 14
```

---

6. Access Nested Dictionary

Problem: From

```
student = {"name": "Ali", "marks": {"math": 80, "science": 90}}
```

print only `"science"` mark.

✅ **Answer:**

```
student = {"name": "Ali", "marks": {"math": 80, "science": 90}}
print(student["marks"]["science"])
```

Output:

```
90
```

---

7. Word Frequency

Problem: Count words in `"this is is a test"`.

✅ **Answer:**

```
text = "this is is a test"
words = text.split()
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1
print(freq)
```

Output:

```
{'this': 1, 'is': 2, 'a': 1, 'test': 1}
```

---

8. Find Largest Number

Problem: Find largest in `[3,6,2,8]`.

✅ **Answer:**

```
nums = [3, 6, 2, 8]
largest = nums[0]
for n in nums:
    if n > largest:
        largest = n
print("Largest:", largest)
```

Output:

```
Largest: 8
```

---

9. Palindrome Check

Problem: Check if `"madam"` is palindrome.

✅ **Answer:**

```
s = "madam"
if s == s[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

Output:

```
Palindrome
```

---

10. Factorial using Loop

Problem: Find factorial of 5.

✅ **Answer:**

```
num = 5
fact = 1
for i in range(1, num+1):
    fact *= i
print("Factorial:", fact)
```

Output:

```
Factorial: 120
```

👉 Do you want me to **continue with problems 11–20 (Easy Level)** in the same detailed way (explanation + code + output), or should I jump to **Moderate problems (21–35)** next?

Alright 🚀 let's **jump to the Moderate problems (11–20)** with **explanation + code + output**.

🟢 Moderate Level (11–20 with Solutions)

---

11. Nested Dict Update

Problem: Update `"science"` mark to 95.

```
student = {"name": "Ali", "marks": {"math": 80, "science": 90}}
```

✅ **Answer:**

```
student = {"name": "Ali", "marks": {"math": 80, "science": 90}}
student["marks"]["science"] = 95
print(student)
```

Output:

```
{'name': 'Ali', 'marks': {'math': 80, 'science': 95}}
```

---

12. String Compression

Problem: `"aaabbcddd"` → `"a3b2c1d3"`

✅ **Answer:**

```
s = "aaabbcddd"
result = ""
count = 1

for i in range(len(s)-1):
    if s[i] == s[i+1]:
        count += 1
    else:
        result += s[i] + str(count)
        count = 1
result += s[-1] + str(count)
print(result)
```

Output:

```
a3b2c1d3
```

---

13. Prime Numbers (1–50)

Problem: Print prime numbers 1–50.

✅ **Answer:**

```
for num in range(2, 51):
    is_prime = True
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        print(num, end=" ")
```

Output:

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

---

14. Fibonacci Series

Problem: Print first 10 Fibonacci numbers.

✅ **Answer:**

```
a, b = 0, 1
for _ in range(10):
    print(a, end=" ")
    a, b = b, a + b
```

Output:

```
0 1 1 2 3 5 8 13 21 34
```

---

15. Find Minimum in List

Problem: Find smallest in `[10, 3, 7]`.

✅ **Answer:**

```
nums = [10, 3, 7]
minimum = nums[0]
for n in nums:
    if n < minimum:
        minimum = n
print("Min:", minimum)
```

Output:

```
Min: 3
```

---

16. Count Digits in Number

Problem: Count digits in `12345`.

✅ **Answer:**

```
num = 12345
count = 0
while num > 0:
    count += 1
    num //= 10
print("Digits:", count)
```

Output:

```
Digits: 5
```

---

17. Remove Duplicates from List

Problem: `[1, 2, 2, 3, 1]` → `[1, 2, 3]`.

✅ **Answer:**

```
nums = [1, 2, 2, 3, 1]
unique = []
for n in nums:
    if n not in unique:
        unique.append(n)
print(unique)
```

Output:

```
[1, 2, 3]
```

---

18. Reverse Words in String

Problem: `"I love Python"` → `"Python love I"`.

✅ **Answer:**

```
s = "I love Python"
words = s.split()
rev = " ".join(words[::-1])
print(rev)
```

Output:

```
Python love I
```

---

19. Armstrong Number Check

Problem: Check if `153` is Armstrong ($1^3+5^3+3^3=153$).

✅ **Answer:**

```
num = 153
digits = [int(d) for d in str(num)]
total = sum(d**3 for d in digits)
if total == num:
    print("Armstrong Number")
else:
    print("Not Armstrong")
```

Output:

```
Armstrong Number
```

---

20. Multiplication Table

Problem: Print table of `5`.

✅ **Answer:**

```
num = 5
for i in range(1, 11):
    print(f"{num} x {i} = {num*i}")
```

Output:

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

🟢 Moderate Level (21–30 with Solutions)

---

21. Flatten Nested Dict

Problem:

```
data = {"a": {"b": {"c": 1}}}
```

Convert → `{"a.b.c": 1}`

✅ **Answer:**

```
def flatten_dict(d, parent_key="", sep="."):
    items = {}
    for k, v in d.items():
        new_key = parent_key + sep + k if parent_key else k
        if isinstance(v, dict):
            items.update(flatten_dict(v, new_key, sep=sep))
        else:
            items[new_key] = v
    return items

data = {"a": {"b": {"c": 1}}}
print(flatten_dict(data))
```

Output:

```
{'a.b.c': 1}
```

---

22. String Anagram Check

Problem: `"listen"` and `"silent"` → anagrams.

✅ **Answer:**

```
s1 = "listen"
s2 = "silent"

if sorted(s1) == sorted(s2):
    print("Anagrams")
else:
    print("Not Anagrams")
```

Output:

```
Anagrams
```

23. Class – Bank Account

Problem: Create class with `deposit` and `withdraw` methods.

✅ **Answer:**

```
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}, Balance: {self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds")
        else:
            self.balance -= amount
            print(f"Withdrew {amount}, Balance: {self.balance}")

account = BankAccount(100)
account.deposit(50)
account.withdraw(30)
account.withdraw(200)
```

Output:

```
Deposited 50, Balance: 150
Withdrew 30, Balance: 120
Insufficient funds
```

---

24. Find Second Largest Number

Problem: `[4, 7, 1, 9, 3]` → second largest (7).

✅ **Answer:**

```
nums = [4, 7, 1, 9, 3]
unique_nums = list(set(nums))
unique_nums.sort()
print("Second largest:", unique_nums[-2])
```

Output:

```
Second largest: 7
```

---

25. Nested Dict Merge

Problem:

```
d1 = {"a": {"x": 1}}
d2 = {"a": {"y": 2}}

→ {"a": {"x": 1, "y": 2}}
```

✅ **Answer:**

```
def merge_dicts(d1, d2):
    result = d1.copy()
    for k, v in d2.items():
        if k in result and isinstance(result[k], dict) and isinstance(v,
dict):
            result[k] = merge_dicts(result[k], v)
        else:
            result[k] = v
    return result

d1 = {"a": {"x": 1}}
d2 = {"a": {"y": 2}}
print(merge_dicts(d1, d2))
```

Output:

```
{'a': {'x': 1, 'y': 2}}
```

---

26. String Frequency (Characters)

Problem: `"hello"` → `{h:1, e:1, l:2, o:1}`

✅ **Answer:**

```
s = "hello"
freq = {}
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

Output:

```
{'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

---

27. Remove Punctuation from String

Problem: `"hello!!! world??"` → `"hello world"`

✅ **Answer:**

```
import string

s = "hello!!! world??"
clean = "".join(ch for ch in s if ch not in string.punctuation)
print(clean)
```

Output:

```
hello world
```

---

28. Find GCD of Two Numbers

Problem: Find GCD of 54, 24.

✅ **Answer:**

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

print("GCD:", gcd(54, 24))
```

Output:

```
GCD: 6
```

---

29. Check Substring

Problem: `"python"` contains `"tho"`?

✅ **Answer:**

```
s = "python"
sub = "tho"
if sub in s:
    print("Substring found")
else:
    print("Not found")
```

Output:

```
Substring found
```

---

30. Class – Student Report

Problem: Store marks as dict & calculate average.

✅ **Answer:**

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks  # dictionary of subject: mark

    def average(self):
        return sum(self.marks.values()) / len(self.marks)

student = Student("Ali", {"math": 80, "science": 90, "english": 70})
print("Average Marks:", student.average())
```

Output:

```
Average Marks: 80.0
```

Problem 40: Sum of Digits Until Single Digit

Write a program to repeatedly sum the digits of a number until you get a single digit.

Example:
Input: `9875` → `9+8+7+5 = 29` → `2+9 = 11` → `1+1 = 2`

✅ **Solution:**

```
def digital_root(n):
    while n >= 10:
        n = sum(int(d) for d in str(n))
    return n

print(digital_root(9875))  # Output: 2
```

Problem 41: Count Frequency of Words

Write a program to count the frequency of each word in a string.

Example:
Input: `"apple banana apple orange banana apple"`

✅ **Solution:**

```
def word_frequency(text):
    words = text.split()
    freq = {}
    for word in words:
        freq[word] = freq.get(word, 0) + 1
    return freq

print(word_frequency("apple banana apple orange banana apple"))
# Output: {'apple': 3, 'banana': 2, 'orange': 1}
```

Problem 42: Reverse Words in a Sentence

Write a program to reverse the words but keep their order.

Example:
Input: `"Hello World"` → Output: `"olleH dlroW"`

✅ **Solution:**

```
def reverse_words(sentence):
    return " ".join(word[::-1] for word in sentence.split())

print(reverse_words("Hello World"))  # Output: olleH dlroW
```

Problem 43: Pascal's Triangle

Print the first `n` rows of Pascal's triangle.

✅ **Solution:**

```
def pascal_triangle(n):
    triangle = [[1]]
    for i in range(1, n):
        prev = triangle[-1]
        row = [1] + [prev[j] + prev[j+1] for j in range(len(prev)-1)] + [1]
        triangle.append(row)
    return triangle
```

```
for row in pascal_triangle(5):
    print(row)

# Output:
# [1]
# [1, 1]
# [1, 2, 1]
# [1, 3, 3, 1]
# [1, 4, 6, 4, 1]
```

Problem 44: Check Balanced Parentheses

Write a program to check if parentheses `()[]{}` are balanced.

✅ **Solution:**

```
def is_balanced(s):
    stack = []
    pairs = {')': '(', '}': '{', ']': '['}
    for ch in s:
        if ch in "({[":
            stack.append(ch)
        elif ch in ")}]":
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
    return not stack

print(is_balanced("{[()]}"))  # True
print(is_balanced("{[(])}"))  # False
```

Problem 45: Find All Prime Numbers in Range

Find all prime numbers between `1` and `n`.

✅ **Solution:**

```
def primes_in_range(n):
    primes = []
    for i in range(2, n+1):
        for j in range(2, int(i**0.5)+1):
            if i % j == 0:
                break
        else:
            primes.append(i)
    return primes

print(primes_in_range(30))
# Output: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

Problem 46: Count Vowels and Consonants

Count vowels and consonants in a string.

✅ **Solution:**

```
def count_vowels_consonants(s):
    vowels = "aeiouAEIOU"
    v = sum(1 for ch in s if ch in vowels)
    c = sum(1 for ch in s if ch.isalpha() and ch not in vowels)
    return v, c

print(count_vowels_consonants("Hello World"))
# Output: (3, 7)
```

Problem 47: Flatten a Nested List

Convert nested list into a single flat list.

✅ **Solution:**

```
def flatten(lst):
    flat = []
    for i in lst:
        if isinstance(i, list):
            flat.extend(flatten(i))
        else:
            flat.append(i)
    return flat

print(flatten([1, [2, [3, 4], 5], 6]))
# Output: [1, 2, 3, 4, 5, 6]
```

Problem 48: LRU Cache Implementation

Implement a simple Least Recently Used (LRU) cache.

✅ **Solution:**

```
class LRUCache:
    def __init__(self, capacity: int):
        self.cache = {}
        self.order = []
        self.capacity = capacity

    def get(self, key):
        if key in self.cache:
            self.order.remove(key)
```

```
            self.order.append(key)
            return self.cache[key]
        return -1

    def put(self, key, value):
        if key in self.cache:
            self.order.remove(key)
        elif len(self.cache) == self.capacity:
            oldest = self.order.pop(0)
            del self.cache[oldest]
        self.cache[key] = value
        self.order.append(key)

cache = LRUCache(2)
cache.put(1, 1)
cache.put(2, 2)
print(cache.get(1))  # 1
cache.put(3, 3)      # Removes key 2
print(cache.get(2))  # -1
```

Problem 49: Find Missing Number

Find the missing number in a list of `1...n`.

✅ **Solution:**

```
def find_missing(arr, n):
    total = n * (n + 1) // 2
    return total - sum(arr)

print(find_missing([1, 2, 4, 5, 6], 6))
# Output: 3
```

Problem 50: Sudoku Validator

Check if a given Sudoku board is valid.

✅ **Solution:**

```
def is_valid_sudoku(board):
    seen = set()
    for i in range(9):
        for j in range(9):
            num = board[i][j]
            if num != ".":
                if (i, num) in seen or (num, j) in seen or (i//3, j//3, num)
in seen:
                    return False
                seen.add((i, num))
                seen.add((num, j))
```

```
                seen.add((i//3, j//3, num))
    return True

board = [
 ["5","3",".",".","7",".",".",".",."],
 ["6",".",".","1","9","5",".",".",."],
 [".","9","8",".",".",".",".","6",."],
 ["8",".",".",".","6",".",".",".","3"],
 ["4",".",".","8",".","3",".",".","1"],
 ["7",".",".",".","2",".",".",".","6"],
 [".","6",".",".",".",".","2","8",."],
 [".",".",".","4","1","9",".",".","5"],
 [".",".",".",".","8",".",".","7","9"]
]
print(is_valid_sudoku(board))  # True
```